

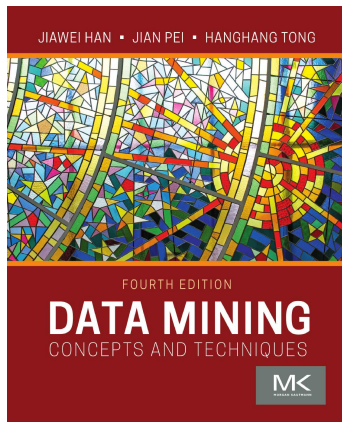
Emergent Digital Technologies

Artificial Neural Networks (ANN)

Jiawei Han & Jian Pei & Hanghang Tong

7 de mayo de 2026

Data Mining: Concepts and Techniques



Content has been extracted from *Data Mining: Concepts and Techniques*, by Jiawei Han & Jian Pei & Hanghang Tong. Morgan Kaufmann. 2022. Visit

<https://shop.elsevier.com/books/data-mining/han/978-0-12-811760-6>.

Introduction to Artificial Neural Networks

- ▶ Artificial Neural Networks (ANNs) are a family of techniques based on biological neural systems.
- ▶ A neural network is a set of connected input-output units where each connection has an associated weight.
- ▶ During learning, the network adjusts these weights to predict correct target values for input data.
- ▶ Applications include computer vision, natural language processing, and machine translation.

Introduction to Artificial Neural Networks (Cont.)

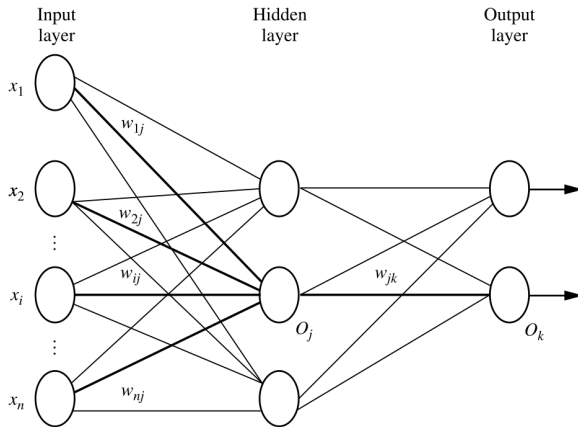


FIGURE 10.1

Multilayer feed-forward neural network.

What is a Unit?

- ▶ The basic building block of an ANN is the **unit**.
- ▶ Historically, basic classifiers like the perceptron and logistic regression are viewed as individual units.
- ▶ A unit is a mathematical function that:
 1. Takes a linear weighted sum of the inputs.
 2. Passes that sum through an activation function $f(\cdot)$.

Mathematical Representation of a Unit

- ▶ Given a data tuple $X = (x_1, x_2, \dots, x_n)$:
- ▶ **Net Input (I):** The linear weighted sum plus a bias scalar b .

$$I = \sum_{i=1}^n x_i w_i + b$$

- ▶ **Output (O):** The result of applying the activation function.

$$O = f(I)$$

- ▶ The bias b acts as a threshold to adjust the net input of the unit.

Mathematical Representation of a Unit (Cont.)

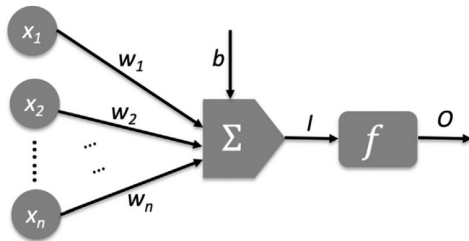


FIGURE 10.2

An illustrative example of unit. Given the inputs $X_1, \dots, X_n$, a unit takes a linear weighted sum and then passes the sum through an activation function $f(\cdot)$. $I = \sum_{i=1}^n X_i w_i + b$ is the net input of the activation function and $O = f(I)$ is the output of the unit. w_i ($i = 1, \dots, n$) are weights and b is the bias scalar.

Activation Functions

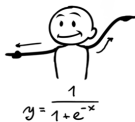
- ▶ Activation functions are typically nonlinear.
- ▶ **Step Function:** Used in perceptrons; outputs 1 if $z \geq n$, else 0.
- ▶ **Sigmoid Function (σ):** Maps input to (0, 1).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- ▶ **ReLU (Rectified Linear Unit):** $f(I) = I$ if $I > 0$, and 0 otherwise.
- ▶ Nonlinearity allows ANNs to model complex, linearly inseparable data.

Activation Functions (Cont.)

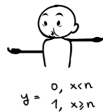
Sigmoid



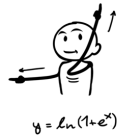
Tanh



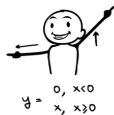
Step Function



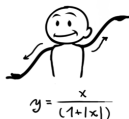
Softplus



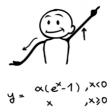
ReLU



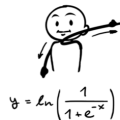
Softsign



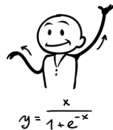
ELU



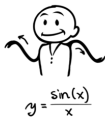
Log of Sigmoid



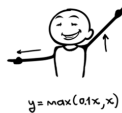
Swish



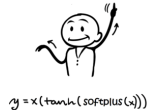
Sinc



Leaky ReLU



Mish



Multilayer Feed-forward Networks

- ▶ This is a powerful and common type of ANN architecture.
- ▶ Consists of an **input layer**, one or more **hidden layers**, and an **output layer**.
- ▶ It is "feed-forward" because weights do not cycle back to previous units or layers.
- ▶ It is "fully connected" if each unit provides input to every unit in the next layer.

Layer Roles in ANN

- ▶ **Input Layer:** Units correspond to the attributes of the input tuple and pass values unchanged.
- ▶ **Hidden Layers:** "Neuron-like" units that perform nonlinear transformations.
- ▶ **Output Layer:** Emits the final network predictions (e.g., class labels or numeric values).
- ▶ A "two-layer" network typically refers to one hidden layer and one output layer (input layer is not counted).

Feature Learning Hierarchy

- ▶ Each layer learns features from the outputs of the previous layer.
- ▶ This creates a hierarchy of learned features at increasing levels of complexity.
- ▶ Example in Computer Vision:
 - ▶ Input: Raw pixels.
 - ▶ Layer 1: Edges.
 - ▶ Layer 2: Contours/Corners.
 - ▶ Layer 3: Object parts (e.g., wheels).

The XOR Problem

- ▶ The XOR problem involves four tuples: $(0,1)$ and $(1,0)$ are positive; $(0,0)$ and $(1,1)$ are negative.
- ▶ This dataset is **linearly inseparable**.
- ▶ A single linear classifier (perceptron) cannot separate the positive and negative tuples.
- ▶ This highlights the need for multilayer architectures with nonlinear activation.

Solving XOR with Neural Networks

- ▶ A two-layer feed-forward network can solve XOR.
- ▶ Two hidden units can act as separate linear classifiers (perceptrons).
- ▶ The output unit combines these “learned features” to create a nonlinear decision boundary.
- ▶ This transforms the feature space into one where the tuples are linearly separable.

Solving XOR with Neural Networks (Cont.)

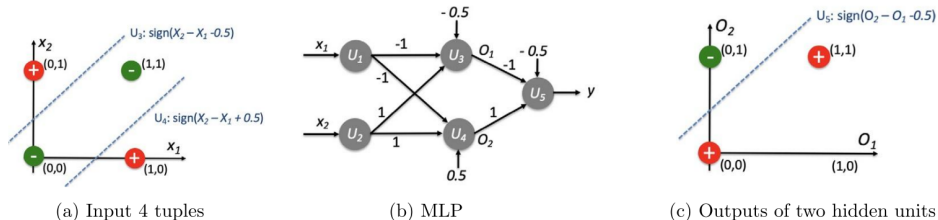


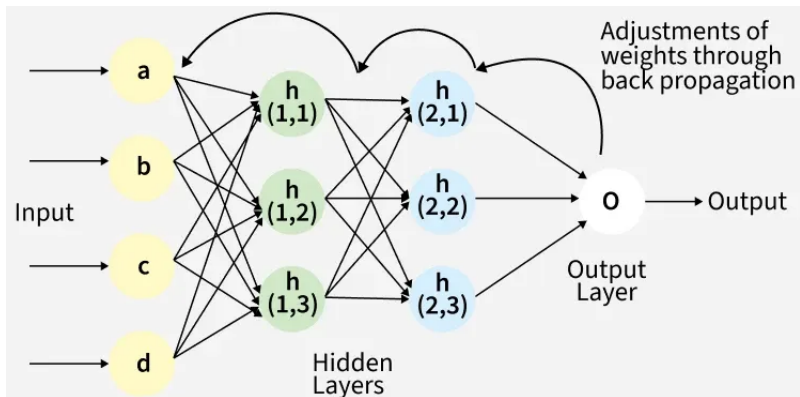
FIGURE 10.3

Solving XOR problem with a two-layer feed-forward neural network with *sign* as the activation function, which is also called multilayer perceptron (MLP). Note that in (c), both negative tuples share the same representation (O_1, O_2) , both being at $(0, 1)$.

The Backpropagation Algorithm

- ▶ A technique to automatically learn weights and biases from training data.
- ▶ It iteratively processes training tuples and compares predictions to known target values.
- ▶ Weights are modified to minimize the loss (e.g., mean-squared error).
- ▶ Modifications are made in the “backward” direction, from the output layer to the first hidden layer.

The Backpropagation Algorithm (Cont.)



<https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/>

Phase 1: Forward Propagation

- ▶ Inputs are fed to the network and pass through the input layer unchanged.
- ▶ For each unit in subsequent layers:
 1. Compute net input: $I_j = \sum_i w_{ij}O_i + b_j$.
 2. Compute output: $O_j = f(I_j)$.
- ▶ Outputs are calculated layer by layer until the output layer provides a prediction.

Phase 2: Backward Propagation of Error

- ▶ The error (δ) is calculated starting from the output layer.
- ▶ For an output unit j :

$$\delta_j = O_j(1 - O_j)(T_j - O_j)$$

- ▶ For a hidden unit j , error is a weighted sum of the errors of the units in the next higher layer (l):

$$\delta_j = O_j(1 - O_j) \sum_l \delta_l w_{jl}$$

Phase 3: Weight and Bias Updates

- ▶ Parameters are updated using a gradient descent method.
- ▶ **Weight Update:** $w_{ij} = w_{ij} - \Delta w_{ij}$

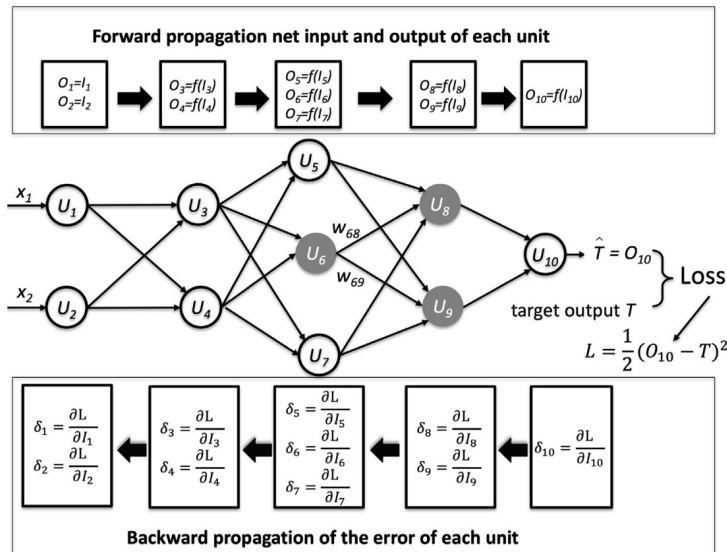
$$\Delta w_{ij} = \eta \delta_j O_i$$

- ▶ **Bias Update:** $b_j = b_j - \Delta b_j$

$$\Delta b_j = \eta \delta_j$$

- ▶ η is the **learning rate**, usually between 0.0 and 1.0.

The Backpropagation Algorithm (Cont.)



Weight Initialization

- ▶ Weights and biases are initialized to small random numbers.
- ▶ **Breaking Symmetry:** It is critical that initial weights for units in the same layer are different.
- ▶ If weights are identical, units will always update in the same way.
- ▶ Random initialization ensures units can learn different features.

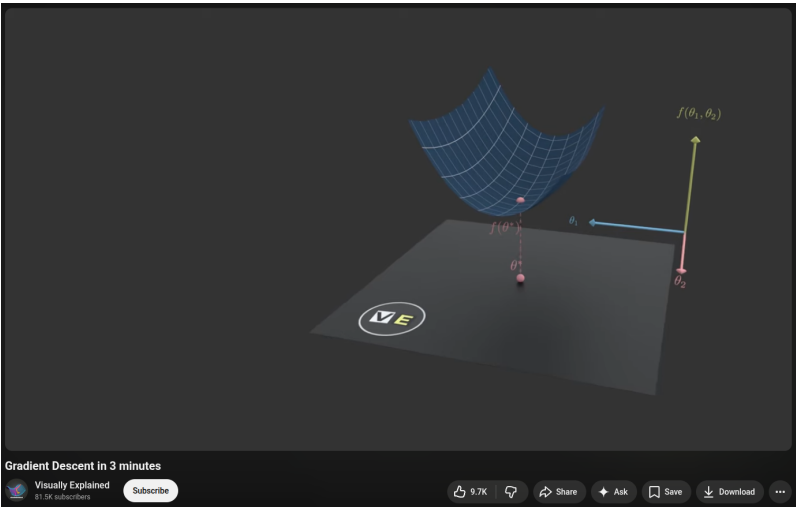
The Learning Rate (η)

- ▶ The learning rate determines the size of the steps taken toward optimal weights.
- ▶ **Too Small:** Training is very slow.
- ▶ **Too Large:** Oscillation between inadequate solutions may occur.
- ▶ A common rule of thumb is to set $\eta = 1/t$, where t is the number of iterations.

Gradient Descent vs. Stochastic Gradient Descent

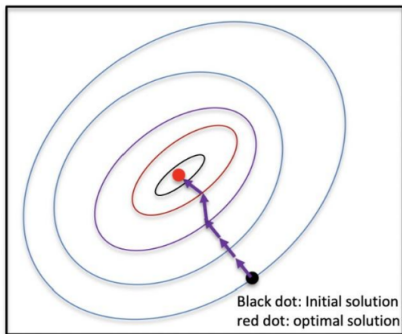
- ▶ **Gradient Descent (Full Batch):** Uses all training tuples to compute the gradient.
- ▶ **Stochastic Gradient Descent (SGD):** Updates parameters using a small “mini-batch” of tuples.
- ▶ SGD finds a direction that approximates the best direction.
- ▶ Overall, SGD is often much more computationally efficient.

Gradient Descent vs. Stochastic Gradient Descent (Cont.)

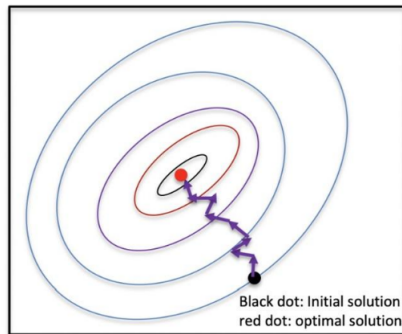


<https://www.youtube.com/watch?v=qg4PchTECck>

Gradient Descent vs. Stochastic Gradient Descent (Cont.)



(a) Gradient descent



(b) Stochastic gradient descent

Matrix Form Representation

- ▶ Neural networks can be represented concisely using matrices.
- ▶ A layer transformation can be written as an **affine transformation**:

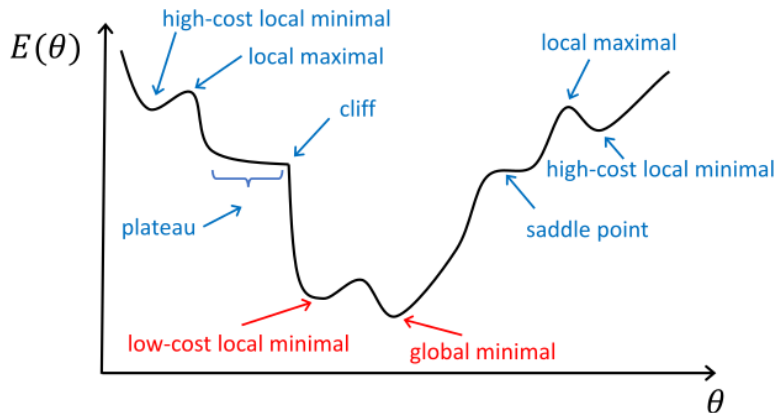
$$h_i = f(W_i h_{i-1} + b_i)$$

- ▶ W_i is the weight matrix and b_i is the bias vector.
- ▶ Matrix multiplication is often accelerated using GPUs instead of CPUs.

Optimization Challenges

- ▶ Training error functions in ANNs are almost always **nonconvex**.
- ▶ Potential pitfalls include:
 - ▶ **Local Minima:** Ending up in a high-cost area.
 - ▶ **Plateaus:** Areas where the gradient is near zero.
 - ▶ **Cliffs:** Areas with very large gradients.
 - ▶ **Saddle Points:** Gradients are zero but it is not a local minimum.

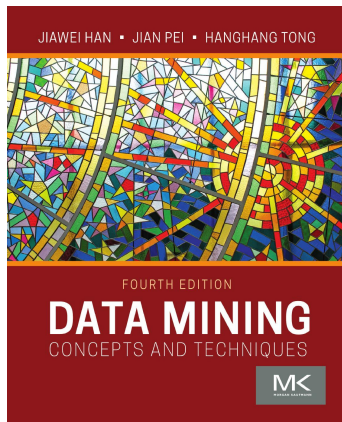
Optimization Challenges (Cont.)



Generalization and Overfitting

- ▶ The goal is to minimize the generalization error on unseen test tuples.
- ▶ **Overfitting:** Model fits training data perfectly but performs poorly on test data.
- ▶ Likely to happen with high model complexity and limited training samples.
- ▶ Effective training requires balancing optimization and generalization.

Data Mining: Concepts and Techniques



Content has been extracted from *Data Mining: Concepts and Techniques*, by Jiawei Han & Jian Pei & Hanghang Tong. Morgan Kaufmann. 2022. Visit

<https://shop.elsevier.com/books/data-mining/han/978-0-12-811760-6>.